

LỘ TRÌNH ĐẠT ĐIỂM TỐI ĐA (A+) ĐỒ ÁN CLOUD

Đề tài 13: Triển khai ứng dụng Container (Manga PMS) trên nền tảng Điện toán Đám mây

Môn học: Điện toán đám mây
(Cloud Computing)

Công nghệ: Docker, Docker Compose, PHP 8.2,
MySQL 8.0, AWS/GCP/Azure

Trạng thái hiện tại: Đã hoàn thành 65%
(Local Docker 100%)

1. BẢNG TIÊU CHÍ CHẤM ĐIỂM CỦA GIẢNG VIÊN

THANG ĐIỂM 10

Tiêu chí đánh giá	Yêu cầu chi tiết từ Giảng viên	Trọng số	Tình trạng
1. Ứng dụng thực tế & Đóng gói	Web app hoàn chỉnh (Manga PMS); viết Dockerfile chuẩn; multi-container Docker Compose.	25% (2.5 đ)	ĐÃ ĐẠT (100%)
2. Tính nhất quán & Bền vững	Cơ chế Volume (Database, Uploads); biến môi trường; chạy độc lập không phụ thuộc máy local.	20% (2.0 đ)	ĐÃ ĐẠT (100%)
3. Triển khai Cloud thực tế	Deploy lên máy ảo Cloud (AWS EC2 / GCP / Azure); cấu hình Security Group / Firewall; mở Public IP.	25% (2.5 đ)	BƯỚC TIẾP THEO
4. Kiểm thử & Phân tích lợi ích	Test chức năng trên Cloud; đo latency/TTFB; so sánh lợi ích Container vs Máy ảo truyền thống (VM).	15% (1.5 đ)	BƯỚC TIẾP THEO
5. Báo cáo & Thuyết trình	Báo cáo kỹ thuật đủ 5 chương; sơ đồ kiến trúc Cloud; trả lời trôi chảy câu hỏi phản biện.	15% (1.5 đ)	BƯỚC TIẾP THEO

2. LỘ TRÌNH 5 GIAI ĐOẠN ĐẠT ĐIỂM TỐI ĐA

GIAI ĐOẠN 1: MÔI TRƯỜNG PHÁT TRIỂN & ĐÓNG GÓI DOCKER LOCAL

ĐÃ HOÀN THÀNH 100%

- ☒ **Xây dựng ứng dụng hoàn chỉnh:** Manga PMS phân quyền 5 vai trò (Mangaka, Editor, Board, Admin, Assistant), User CRUD, Series Search/Filter/Pagination, Chapter Workflow.
- ☒ **Viết Dockerfile:** Base image php:8.2-apache, cài pdo, pdo_mysql, bật a2enmod rewrite, phân quyền www-data:www-data.
- ☒ **Cấu hình Docker Compose:** Điều phối container manga-pms-app và manga-pms-db (MySQL 8.0) qua bridge network manga-network.
- ☒ **Thiết lập Data Persistence:** Named volume mysql_data cho CSDL và bind-mount ./uploads cho file ảnh/bản thảo.
- ☒ **Tối ưu hóa Caching & Hiệu năng:** Sửa cache CSS tĩnh v=1.0.0, tốc độ phản hồi đạt cực nhanh (~10-20ms).

GIAI ĐOẠN 2: TỐI ƯU CẤU HÌNH & CHUẨN BỊ SẴN SÀNG CLOUD

ĐÃ HOÀN THÀNH 100%

- ☒ **Đọc biến môi trường linh hoạt:** File config/database.php tự nhận diện biến môi trường DB_HOST=db khi chạy Docker và fallback về 127.0.0.1 khi chạy XAMPP.
- ☒ **Healthcheck tự động:** Cấu hình healthcheck cho MySQL để PHP app chỉ start khi DB đã sẵn sàng kết nối.
- ☒ **Tự động nạp Schema & Dữ liệu mẫu (Seed Data):** Đã cấu hình các script SQL tự động khởi tạo trong thư mục docker-entrypoint-initdb.d/.

GIAI ĐOẠN 3: TRIỂN KHAI THỰC TẾ LÊN CLOUD (CLOUD DEPLOYMENT)

 BƯỚC TRỌNG TÂM CẦN LÀM

- ☐ **Bước 3.1: Đẩy mã nguồn lên GitHub / GitLab:** Đảm bảo `.dockerignore` và `.gitignore` sạch sẽ.
- ☐ **Bước 3.2: Khởi tạo Máy ảo Cloud (IaaS):**
 - Lựa chọn 1 trong các nền tảng: **AWS EC2** (t2.micro Free Tier), **Google Cloud (GCP) Compute Engine**, **Azure Virtual Machine**, hoặc **DigitalOcean Droplet**.
 - Chọn Hệ điều hành: **Ubuntu 22.04 / 24.04 LTS**.
- ☐ **Bước 3.3: Cấu hình Firewall / Security Group trên Cloud:** Mở các cổng mạng Inbound: **Port 80 (HTTP)**, **Port 443 (HTTPS)**, **Port 8080 (App)**, **Port 22 (SSH)**.
- ☐ **Bước 3.4: Cài đặt Docker trên Máy ảo Cloud:** Chạy script cài đặt nhanh: `curl -fsSL https://get.docker.com | sh`.
- ☐ **Bước 3.5: Kéo Code & Khởi chạy ứng dụng:** Clone repo Git về Cloud và chạy đúng 1 câu lệnh duy nhất: `docker compose up -d --build`.

Mẹo ghi điểm tuyệt đối: Quay video ngắn 1-2 phút chứng minh quá trình deploy từ lúc gõ lệnh trên Cloud đến khi web chạy thành công trên Public IP.

GIẢI ĐOẠN 4: KIỂM THỬ TRÊN CLOUD & ĐO LƯỜNG HIỆU NĂNG

 CẦN THỰC HIỆN

- ☐ **Kiểm tra truy cập Internet:** Mở trình duyệt ngoài mạng nội bộ, truy cập `http://[PUBLIC_IP_CLOUD]:8080`.
- ☐ **Kiểm thử chức năng đầy đủ:** Đăng nhập, thêm truyện, upload ảnh bìa, nộp bản thảo, duyệt chapter.
- ☐ **Thực nghiệm Data Persistence:** Dừng container trên Cloud (`docker compose down`) rồi bật lại (`docker compose up -d`) để chứng minh dữ liệu và ảnh upload vẫn còn nguyên vẹn.
- ☐ **Đo lường hiệu năng & tài nguyên Cloud:** Dùng lệnh `docker stats` để chụp ảnh mức tiêu thụ RAM (<30MB) và CPU (<1%) trên Cloud.

GIẢI ĐOẠN 5: HOÀN THIỆN BÁO CÁO MÔN HỌC & SLIDE THUYẾT TRÌNH

 CẦN THỰC HIỆN

- ☐ **Soạn thảo Báo cáo Đồ án (5 Chương chuẩn học phần):**
 - *Chương 1:* Tổng quan đề tài, bài toán Manga PMS và mục tiêu Container hóa.
 - *Chương 2:* Cơ sở lý thuyết về Docker, Docker Compose, Kiến trúc Microservices/Container, So sánh VM vs Container.
 - *Chương 3:* Thiết kế hệ thống & Đóng gói ứng dụng (Phân tích Dockerfile, Docker Compose, Volume, Network).
 - *Chương 4:* Quá trình Triển khai trên nền tảng Cloud (AWS/GCP/Azure), Cấu hình Network & Firewall.
 - *Chương 5:* Kết quả thử nghiệm, Đo lường hiệu năng, Phân tích lợi ích thực tiễn và Kết luận.
- ☐ **Thiết kế Slide thuyết trình:** 12-15 slide trực quan, có sơ đồ kiến trúc, ảnh chụp màn hình Cloud thực tế.

3. BỘ CÂU HỎI PHẢN BIỆN THƯỜNG GẶP CỦA GIẢNG VIÊN (CÁCH TRẢ LỜI ĐẠT ĐIỂM TỐI ĐA)

? Câu 1: Tại sao nhóm lại sử dụng Docker Compose thay vì chạy từng container bằng lệnh "docker run"?

💡 Trả lời: Docker Compose cho phép định nghĩa kiến trúc đa container (App + MySQL) dưới dạng code (IaC - Infrastructure as Code). Nó giúp quản lý tập trung biến môi trường, tự động tạo mạng nội bộ bridge cô lập, quản lý volume và thử tự khởi động (healthcheck) chỉ bằng 1 câu lệnh duy nhất `docker compose up -d`, đảm bảo tính nhất quán tuyệt đối giữa máy dev và máy chủ cloud.

? Câu 2: Khi container bị xóa (Recreate / Destroy), dữ liệu CSDL và file upload có bị mất không? Nhóm giải quyết thế nào?

💡 Trả lời: Hoàn toàn không mất dữ liệu. Nhóm đã tách biệt tầng Stateless (PHP source code) và tầng Stateful (Dữ liệu) bằng cơ chế Docker Volume: CSDL MySQL được lưu trên named volume `mysql_data:/var/lib/mysql`, file upload được gắn bind-mount `./uploads:/var/www/html/uploads`. Vòng đời của dữ liệu tồn tại độc lập với vòng đời của container.

? Câu 3: So sánh lợi ích cốt lõi giữa việc triển khai ứng dụng bằng Docker Container so với Máy ảo truyền thống (VMware/VirtualBox) trên Cloud?

💡 Trả lời:

1. *Tài nguyên:* Container chia sẻ chung Linux Kernel của Host nên cực kỳ nhẹ (App PHP chỉ tốn ~20MB RAM so với 2-4GB của máy ảo đầy đủ OS).
2. *Tốc độ khởi động:* Container khởi động trong vài giây thay vì vài phút như VM.
3. *Tính linh hoạt (Portability):* Đóng gói toàn bộ runtime, library, dependencies giúp chạy nhất quán ở bất kỳ nền tảng Cloud nào mà không bị lỗi môi trường.

? Câu 4: Nếu lượng người dùng truy cập tăng đột biến (Traffic Spike), giải pháp mở rộng (Scale) trên Cloud là gì?

💡 Trả lời: Có thể mở rộng theo chiều ngang (Horizontal Scaling): Tăng số lượng container app bằng lệnh `docker compose up --scale app=3 -d`, đặt thêm một Load Balancer (Nginx hoặc AWS ALB) ở phía trước để phân tải; hoặc chuyển đổi sang dịch vụ điều phối container quy mô lớn như Docker Swarm / Kubernetes (AWS EKS, GCP GKE).